



Bodensee Dogs

Germany



Rescue Maze

Team

We are a team of four students from the Lake Constance region in southern Germany and are members of the robotics club at our school. Robotics has become an integral part of our everyday life, which we are committed to not only on school days, but also on weekends and vacations. Over the past few years, we have gained valuable knowledge and strengthened our teamwork. We highly value the importance of a strong team spirit, with each member contributing their unique strengths, and are very proud to be part of RoboCupJunior.



Laurence Schreiter

CAD and mechanical design, Team Captain

"I designed and fabricated all the robot components, including the suspension system consisting of silicone wheels and two movable axles."



Finn ten Tusscher

PCB design, electronics developer

"I am responsible for the design, the production, and the assembly of the robot's main, power, UI and ToF PCBs."



Jannik von Bank

Sequential control, logging system

"I integrated the sequential control system with which the robot drives and performs the different tasks. Additionally, I coded our custom logging system."



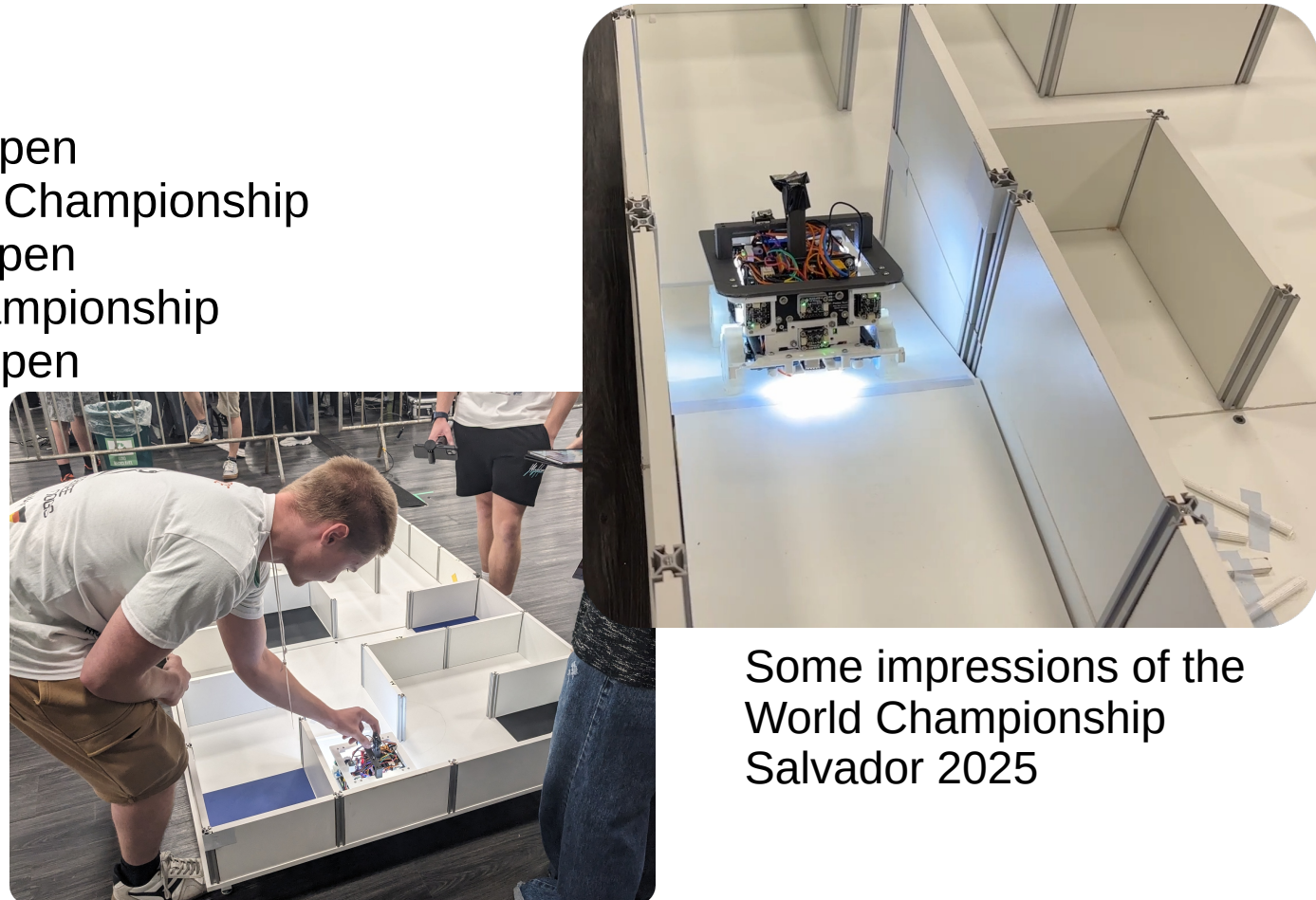
Thilo Hirscher

Navigation and map visualization, Victim identification

"I am responsible for the navigation and the map, as well as the custom victim identification algorithms. I also developed the visualization application for the multi-floor map."

Previous Achievements

- 2024: 4th place at German Open
- 2024: 2nd place at European Championship
- 2025: 1st place at German Open
- 2025: 3rd place at World Championship
- 2026: 2nd place at German Open



Some impressions of the World Championship Salvador 2025

Contact

We share our work on many different social media platforms regularly. You can find the links to those on the website of our robotics club at our school, the Bildungszentrum Markdorf <https://markdorf-robotics.de/> or simply scan the QR code.

Address

Roboter AG Bildungszentrum Markdorf
Ensisheimerstraße 30
D-88677 Markdorf, Germany

E-Mail

bodenseedogs@gmail.com



PCBs

All PCBs are designed, assembled and soldered by hand.

Main PCB (under the chassis)

Our main PCB houses the Teensy 4.0, a fast and small microcontroller. It also connects our OpenMV cameras and includes the BNO055 gyro sensor.

Power PCB

This PCB is connected to the battery and supplies the robot with power. It also integrates our motor drivers and provides the PWM output for our servo.

UI PCB

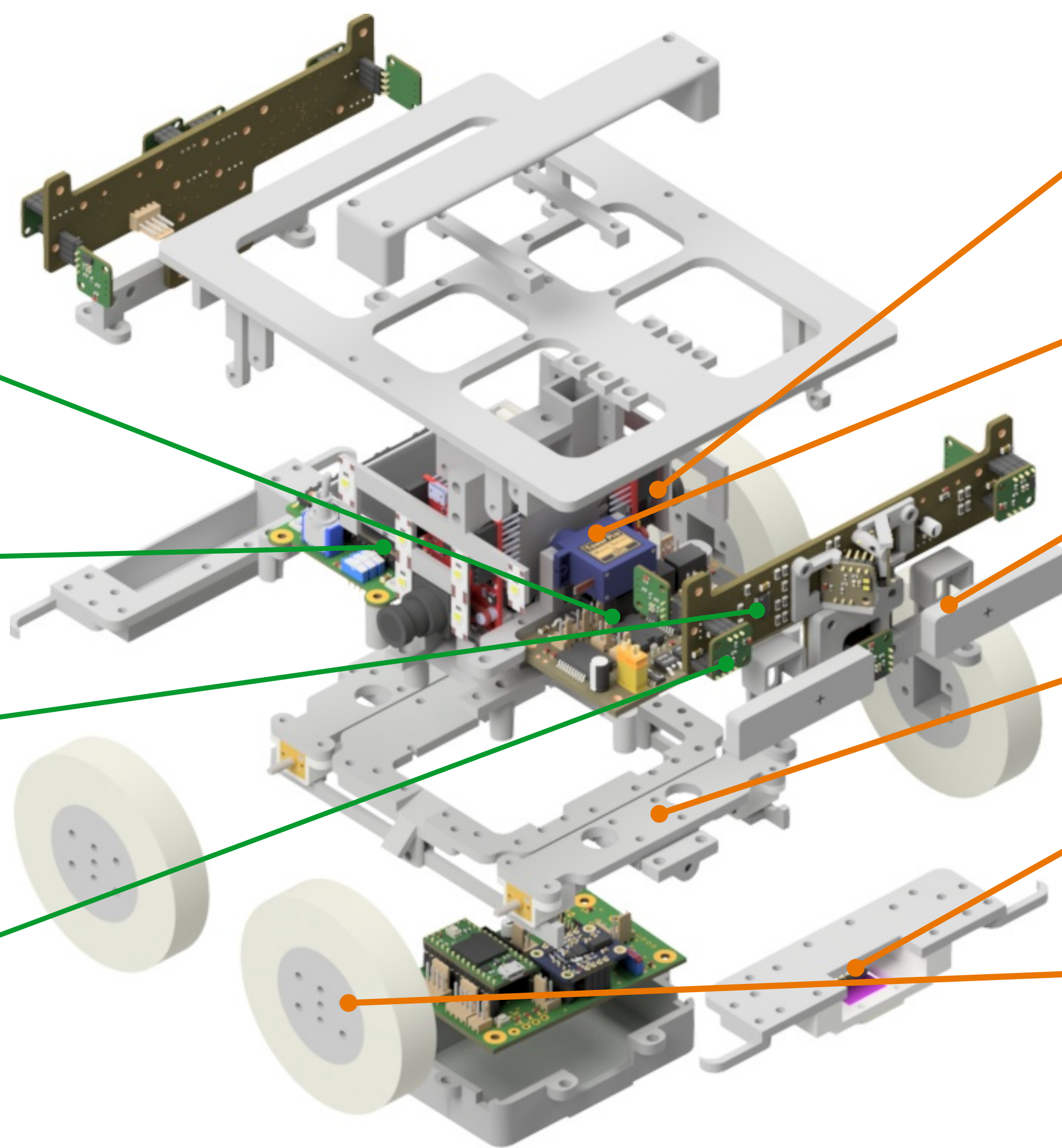
It contains our display, switches, buttons and LEDs for debugging purposes and clear indications during a run.

ToF PCB

In order to prevent cable clutter and plug issues, we designed ToF PCBs to house all ToFs and the color sensor. As all ToFs have the same non-modifiable I2C address the PCB is equipped with a Multiplexer.

ToF VL53L4CD Breakouts

Our ToF breakouts are designed asymmetrically to maximize the distance between two adjacent sensors. Therefore the robot can evaluate its position fast and accurately in the maze.



OpenMV N6 Cameras

We use two cameras that operate independently from the Teensy 4.0 to ensure high frame rates. Both cameras are equipped with fisheye lenses, to handle the narrow distance to the wall.

Rescue kit deployment

We use a metal gear SG90 Servo to control the direction of the deployment. The torque of the servo is transferred to a toothed rack to guarantee linear trajectory.

Limit Switches

We have two switches at the front of the robot that are triggered when the robot is driving against an object. This prevents major offsets of the robot's position.

Suspension system

The front and back wheels are mounted on a movable axle. Both axles are connected with a linkage.

Color Sensor

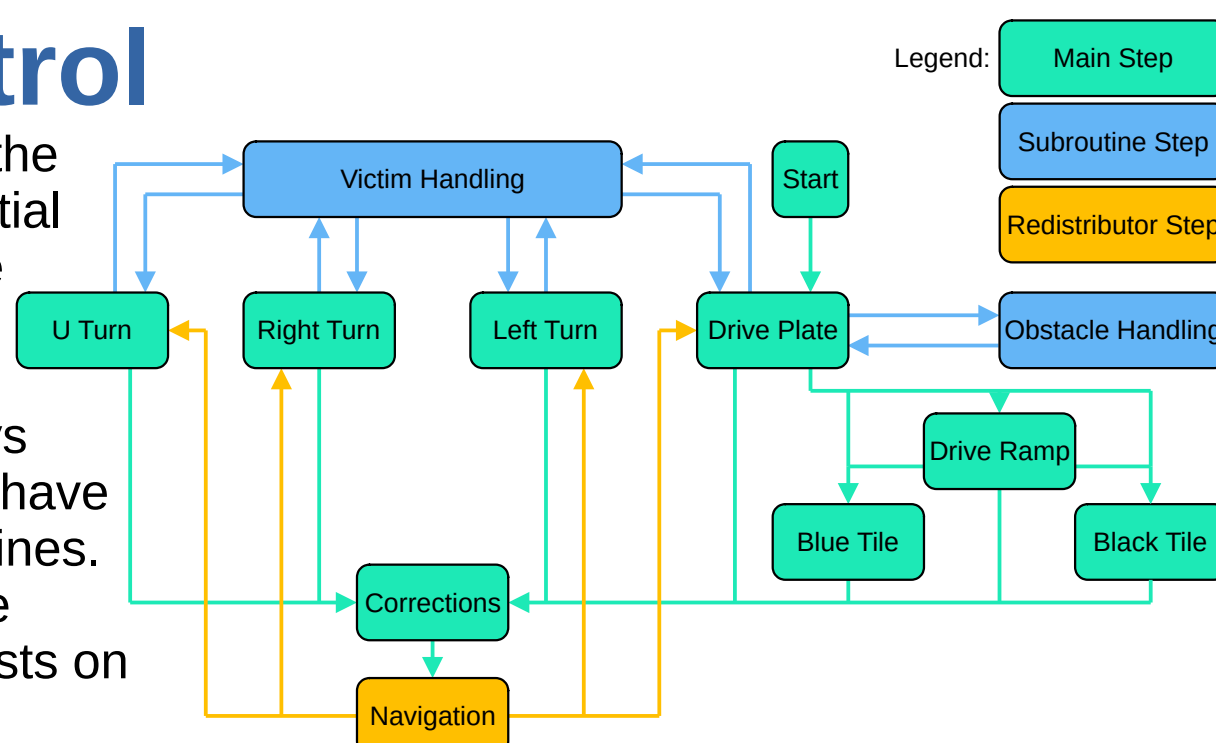
We use the Adafruit TCS34725 to detect the different tile colors. The strong illumination ensures clear distinction between the colors.

Silicone Wheels

Self-made wheels allow us to fully customize the size and profile. The soft silicone guarantees high grip and smooth traversing over ramps, stairs, bumpers and debris.

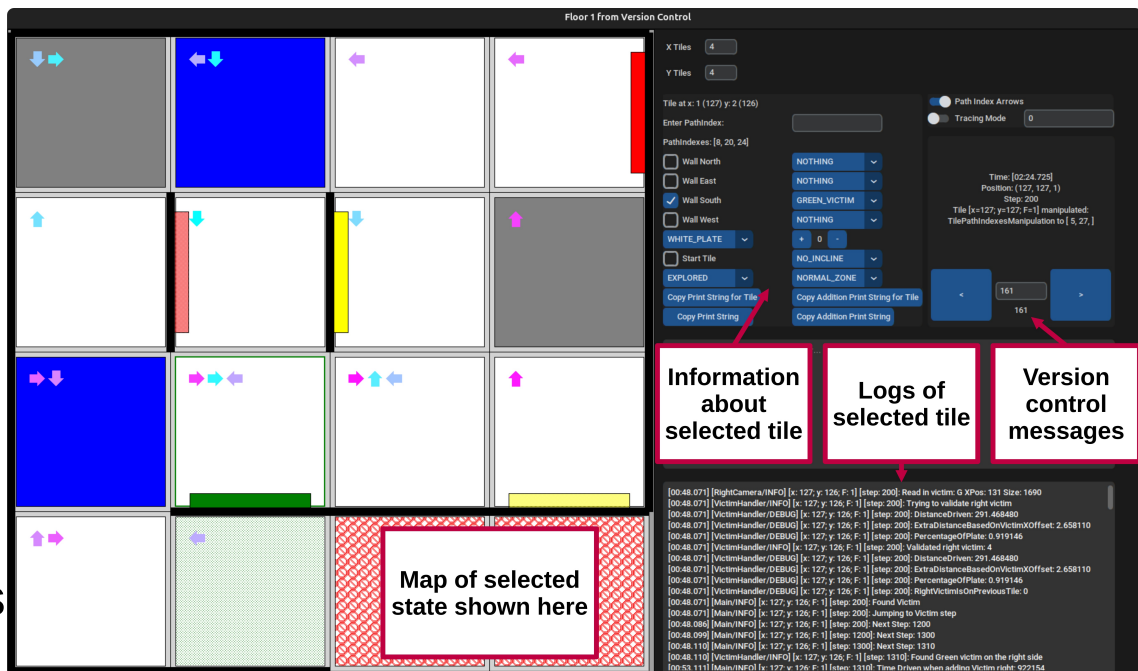
Sequential control

The foundation of our program is the IPO model combined with sequential control. To create a clear structure the control system is divided into three types of steps, as shown on the right. Main steps should always lead to a distributor step and also have the option of branching to subroutines. Additionally this allows us to reuse subroutines or perform isolated tests on individual or sequences of steps.



SD Card Logging

For fast and easy debugging we use a custom logger. Every log message is regularly saved to an SD card in combination with the current map state as well as something we call version control messages. Every time the map is changed a version control message is created to store this difference and later saved to the SD card. Furthermore we have ensured that even if the robot crashes we still flush all information to the SD card using a custom crash handler. To view all this data we developed a map visualization tool that can display the version control messages as a map allowing us to view all the changes to the map and see what log messages were logged on a selected tile.



We use and the dev. environment

VS Code + Platform IO

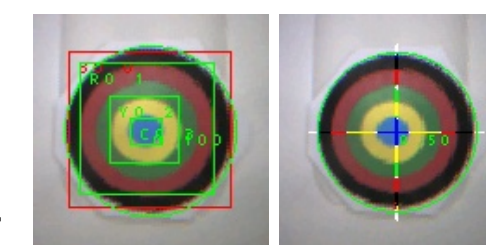
We use and the dev. environment

OpenMV IDE

Victim Identification

Cognitive Targets

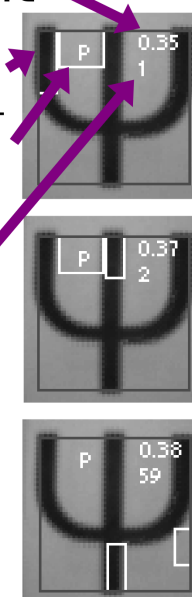
are identified using a custom algorithm, which starts by finding, filtering and grouping color blobs. Then all pixels from the center outward get assigned a color using a 1-NN classifier. This data can be used to evaluate the thickness of the color rings.



Quotient

Black rect
White rect

Feature Number



Visual Victims

1. Find black blobs with the find_blobs() function.
 2. Filter by size and position. Calculate rotation using axis lines.
 3. Apply letter-specific Haar-cascade like feature collections.
- A feature compares two areas of a blob based on their brightness. If the brightness quotient is below a certain threshold, the next feature is checked; otherwise, the blob gets discarded. For example, if all the features of the letter Ψ feature collection passed, the letter Ψ is identified. We implemented the Haar-cascade like system directly in the firmware of the camera to ensure high frame rates.

Navigation

LOP Behavior

When an LOP is triggered the map gets rebuilt to the last matching checkpoint using the version control messages of the map. This ensures that the map will be exactly the same as at the checkpoint. Also the tile on which the LOP was triggered gets flagged as unsafe, which ensures that the robot only drives over the tile when there is no other path.

Unit testing

With Googletest and our map visualization tool to build test maps, we wrote over 210 fully independent unit tests to test the navigation and mapping code thoroughly.

